

16.2-Random_Forest_Regression

August 26, 2025

1 Random Forest Regression Implementation

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
[2]: df = pd.read_csv("../0-Dataset/cardekho_imputed.csv", index_col = [0])
```

```
[3]: df.head()
```

```
[3]:
```

	car_name	brand	model	vehicle_age	km_driven	seller_type	\
0	Maruti Alto	Maruti	Alto	9	120000	Individual	
1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	
2	Hyundai i20	Hyundai	i20	11	60000	Individual	
3	Maruti Alto	Maruti	Alto	9	37000	Individual	
4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer	

	fuel_type	transmission_type	mileage	engine	max_power	seats	\
0	Petrol	Manual	19.70	796	46.30	5	
1	Petrol	Manual	18.90	1197	82.00	5	
2	Petrol	Manual	17.00	1197	80.00	5	
3	Petrol	Manual	20.92	998	67.10	5	
4	Diesel	Manual	22.77	1498	98.59	5	

	selling_price
0	120000
1	550000
2	215000
3	226000
4	570000

```
[4]: df.isnull().sum()
```

```
[4]: car_name      0
      brand        0
      model        0
      vehicle_age  0
      km_driven    0
      seller_type  0
      fuel_type    0
      transmission_type  0
      mileage      0
      engine       0
      max_power    0
      seats        0
      selling_price 0
      dtype: int64
```

```
[5]: df.drop(['car_name', 'brand'], axis=1, inplace=True)
```

```
[6]: df.head()
```

```
[6]:      model  vehicle_age  km_driven  seller_type  fuel_type  transmission_type  \
0      Alto             9     120000  Individual    Petrol             Manual
1      Grand             5      20000  Individual    Petrol             Manual
2       i20             11     60000  Individual    Petrol             Manual
3      Alto             9      37000  Individual    Petrol             Manual
4  Ecosport             6      30000    Dealer    Diesel             Manual

      mileage  engine  max_power  seats  selling_price
0      19.70     796     46.30     5         120000
1      18.90    1197     82.00     5         550000
2      17.00    1197     80.00     5         215000
3      20.92     998     67.10     5         226000
4      22.77    1498     98.59     5         570000
```

```
[7]: df['model'].unique()
```

```
[7]: array(['Alto', 'Grand', 'i20', 'Ecosport', 'Wagon R', 'i10', 'Venue',
        'Swift', 'Verna', 'Duster', 'Cooper', 'Ciaz', 'C-Class', 'Innova',
        'Baleno', 'Swift Dzire', 'Vento', 'Creta', 'City', 'Bolero',
        'Fortuner', 'KWID', 'Amaze', 'Santro', 'XUV500', 'KUV100', 'Ignis',
        'RediGO', 'Scorpio', 'Marazzo', 'Aspire', 'Figo', 'Vitara',
        'Tiago', 'Polo', 'Seltos', 'Celerio', 'GO', '5', 'CR-V',
        'Endeavour', 'KUV', 'Jazz', '3', 'A4', 'Tigor', 'Ertiga', 'Safari',
        'Thar', 'Hexa', 'Rover', 'Eeco', 'A6', 'E-Class', 'Q7', 'Z4', '6',
        'XF', 'X5', 'Hector', 'Civic', 'D-Max', 'Cayenne', 'X1', 'Rapid',
        'Freestyle', 'Superb', 'Nexon', 'XUV300', 'Dzire VXi', 'S90',
        'WR-V', 'XL6', 'Triber', 'ES', 'Wrangler', 'Camry', 'Elantra',
        'Yaris', 'GL-Class', '7', 'S-Presso', 'Dzire LXI', 'Aura', 'XC',
```

```
'Ghibli', 'Continental', 'CR', 'Kicks', 'S-Class', 'Tucson',
'Harrier', 'X3', 'Octavia', 'Compass', 'CLS', 'redi-GO', 'Glanza',
'Macan', 'X4', 'Dzire ZXI', 'XC90', 'F-PACE', 'A8', 'MUX',
'GTC4Lusso', 'GLS', 'X-Trail', 'XE', 'XC60', 'Panamera', 'Alturas',
'Altroz', 'NX', 'Carnival', 'C', 'RX', 'Ghost', 'Quattroporte',
'Gurkha'], dtype=object)
```

```
[8]: num_features = [feature for feature in df.columns if df[feature].dtype != 'O']
print(f"Numerical Features in dataset: {len(num_features)}")

cat_features = [feature for feature in df.columns if df[feature].dtype == 'O']
print(f"Categorical Features in dataset: {len(cat_features)}")
```

```
Numerical Features in dataset: 7
Categorical Features in dataset: 4
```

```
[9]: from sklearn.model_selection import train_test_split
X = df.drop(['selling_price'], axis=1)
y = df['selling_price']
```

```
[10]: X.head()
```

```
[10]:      model  vehicle_age  km_driven  seller_type  fuel_type  transmission_type  \
0      Alto             9    120000  Individual    Petrol             Manual
1      Grand             5     20000  Individual    Petrol             Manual
2      i20             11    60000  Individual    Petrol             Manual
3      Alto             9     37000  Individual    Petrol             Manual
4  Ecosport             6     30000    Dealer    Diesel             Manual

      mileage  engine  max_power  seats
0     19.70     796     46.30     5
1     18.90    1197     82.00     5
2     17.00    1197     80.00     5
3     20.92     998     67.10     5
4     22.77    1498     98.59     5
```

1.1 Feature Encoding and Scaling

```
[11]: from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder
le = LabelEncoder()
X['model'] = le.fit_transform(X['model'])
```

```
[12]: X.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15411 entries, 0 to 19543
Data columns (total 10 columns):
#   Column              Non-Null Count  Dtype
#   ...
```

```

---  -----
0   model                15411 non-null  int32
1   vehicle_age          15411 non-null  int64
2   km_driven            15411 non-null  int64
3   seller_type          15411 non-null  object
4   fuel_type            15411 non-null  object
5   transmission_type    15411 non-null  object
6   mileage              15411 non-null  float64
7   engine               15411 non-null  int64
8   max_power            15411 non-null  float64
9   seats                15411 non-null  int64
dtypes: float64(2), int32(1), int64(4), object(3)
memory usage: 1.2+ MB

```

```

[13]: # creating column transformer with 3 types of transformers
num_features = X.select_dtypes(exclude="object").columns
onehot_columns = ['seller_type', 'fuel_type', 'transmission_type']
lable_encoder_columns = ['model']

```

```

[14]: from sklearn.compose import ColumnTransformer
numeric_transformer = StandardScaler()
oh_transformer = OneHotEncoder(drop='first')

preprocessor = ColumnTransformer(
    [
        ("OneHotEncoder", oh_transformer, onehot_columns),
        ("StandardScaler", numeric_transformer, num_features)
    ], remainder='passthrough' ## other columns are not deleted just pass
    ↳ the other columns
)

```

```

[15]: X = preprocessor.fit_transform(X)

```

```

[16]: pd.DataFrame(X)

```

```

[16]:
      0      1      2      3      4      5      6      7      8      9  \
0      1.0  0.0  0.0  0.0  0.0  1.0  1.0 -1.519714  0.983562  1.247335
1      1.0  0.0  0.0  0.0  0.0  1.0  1.0 -0.225693 -0.343933 -0.690016
2      1.0  0.0  0.0  0.0  0.0  1.0  1.0  1.536377  1.647309  0.084924
3      1.0  0.0  0.0  0.0  0.0  1.0  1.0 -1.519714  0.983562 -0.360667
4      0.0  0.0  1.0  0.0  0.0  0.0  1.0 -0.666211 -0.012060 -0.496281
...
15406  0.0  0.0  0.0  0.0  0.0  1.0  1.0  1.508844  0.983562 -0.869744
15407  0.0  0.0  0.0  0.0  0.0  1.0  1.0 -0.556082 -1.339555 -0.728763
15408  0.0  0.0  1.0  0.0  0.0  0.0  1.0  0.407551 -0.012060  0.220539
15409  0.0  0.0  1.0  0.0  0.0  0.0  1.0  1.426247 -0.343933  72.541850
15410  0.0  0.0  0.0  0.0  0.0  1.0  0.0 -1.024131 -1.339555 -0.825631

```

	10	11	12	13
0	-0.000276	-1.324259	-1.263352	-0.403022
1	-0.192071	-0.554718	-0.432571	-0.403022
2	-0.647583	-0.554718	-0.479113	-0.403022
3	0.292211	-0.936610	-0.779312	-0.403022
4	0.735736	0.022918	-0.046502	-0.403022
...	...	...	...	...
15406	0.026096	-0.767733	-0.757204	-0.403022
15407	-0.527711	-0.216964	-0.220803	2.073444
15408	0.344954	0.022918	0.068225	-0.403022
15409	-0.887326	1.329794	0.917158	2.073444
15410	-0.407839	0.020999	0.395884	-0.403022

[15411 rows x 14 columns]

```
[17]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳ random_state=42)
```

1.2 Model Training and Model Selection

```
[18]: from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

1.2.1 Creating a function to Evaluate the Model

```
[19]: def evaluate_model(true, predicted):
    mae = mean_absolute_error(true, predicted)
    mse = mean_squared_error(true, predicted)
    rmse = np.sqrt(mean_squared_error(true, predicted))
    r2 = r2_score(true, predicted)

    return mae, mse, r2, rmse
```

1.2.2 Model Training

```
[20]: models = {
    "LinearRegression": LinearRegression(),
    "Lasso": Lasso(),
    "Ridge": Ridge(),
    "K-Neighbors Regressor": KNeighborsRegressor(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest Regressor": RandomForestRegressor()
}
```

```

for i in range(len(list(models))):
    model = list(models.values())[i]
    model.fit(X_train, y_train)

    # make prediction
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    # evaluate the train and test dataset
    model_train_mae, model_train_mse, model_train_r2, model_train_rmse = _
    evaluate_model(y_train, y_train_pred)

    model_test_mae, model_test_mse, model_test_r2, model_test_rmse = _
    evaluate_model(y_test, y_test_pred)

    print(list(models.keys())[i])
    print('Model performance for training set')
    print("- Root Mean Squared Error: {:.4f}".format(model_train_rmse))
    print("- Mean Absolute Error: {:.4f}".format(model_train_mae))
    print("- Mean Squared Error: {:.4f}".format(model_train_mse))
    print("- R2 Score: {:.4f}".format(model_train_r2))

    print("-----")

    print('Model performance for testing set')
    print("- Root Mean Squared Error: {:.4f}".format(model_test_rmse))
    print("- Mean Absolute Error: {:.4f}".format(model_test_mae))
    print("- Mean Squared Error: {:.4f}".format(model_test_mse))
    print("- R2 Score: {:.4f}".format(model_test_r2))

    print('='*35)
    print('\n')

```

LinearRegression

Model performance for training set

- Root Mean Squared Error: 559313.7144
- Mean Absolute Error: 268437.6549
- Mean Squared Error: 312831831093.0353
- R2 Score: 0.6183

Model performance for testing set

- Root Mean Squared Error: 507556.7988
- Mean Absolute Error: 281329.9042
- Mean Squared Error: 257613903997.1092
- R2 Score: 0.6575

=====

Lasso

Model performance for training set

- Root Mean Squared Error: 559313.7247
- Mean Absolute Error: 268436.5960
- Mean Squared Error: 312831842666.6471
- R2 Score: 0.6183

Model performance for testing set

- Root Mean Squared Error: 507555.9012
- Mean Absolute Error: 281329.3313
- Mean Squared Error: 257612992800.0465
- R2 Score: 0.6575

Ridge

Model performance for training set

- Root Mean Squared Error: 559314.5337
- Mean Absolute Error: 268393.2589
- Mean Squared Error: 312832747644.5701
- R2 Score: 0.6183

Model performance for testing set

- Root Mean Squared Error: 507539.4235
- Mean Absolute Error: 281276.8149
- Mean Squared Error: 257596266396.6422
- R2 Score: 0.6575

K-Neighbors Regressor

Model performance for training set

- Root Mean Squared Error: 338135.3446
- Mean Absolute Error: 91849.5411
- Mean Squared Error: 114335511289.5152
- R2 Score: 0.8605

Model performance for testing set

- Root Mean Squared Error: 287631.2919
- Mean Absolute Error: 116557.2502
- Mean Squared Error: 82731760089.8843
- R2 Score: 0.8900

Decision Tree

Model performance for training set

```

- Root Mean Squared Error: 20329.7436
- Mean Absolute Error: 4728.8897
- Mean Squared Error: 413298475.0162
- R2 Score: 0.9995
-----
Model performance for testing set
- Root Mean Squared Error: 387105.9378
- Mean Absolute Error: 133430.8121
- Mean Squared Error: 149851007094.2216
- R2 Score: 0.8007
=====

```

```

Random Forest Regressor
Model performance for training set
- Root Mean Squared Error: 143993.2499
- Mean Absolute Error: 40076.6193
- Mean Squared Error: 20734056002.4591
- R2 Score: 0.9747
-----
Model performance for testing set
- Root Mean Squared Error: 246879.0076
- Mean Absolute Error: 105992.0028
- Mean Squared Error: 60949244389.9353
- R2 Score: 0.9190
=====

```

1.3 Hyperparameter Tuning

```

[21]: knn_params = {'n_neighbors': [2, 3, 10, 20, 40, 50]}
      rf_params = {
          "max_depth": [5, 8, 15, None, 10],
          "max_features": [5, 7, "auto", 8],
          "min_samples_split": [2, 8, 15, 20],
          "n_estimators": [50, 70, 80, 85]
      }

```

```

[22]: randomcv_models = [
      ('KNN', KNeighborsRegressor(), knn_params),
      ('RF', RandomForestRegressor(), rf_params)
]

```

```

[23]: from sklearn.model_selection import RandomizedSearchCV

      model_param = {}

```



```

for name, model, params in randomcv_models:
    random = RandomizedSearchCV(estimator=model,
                                param_distributions=params,
                                n_iter=100,
                                cv=3,
                                verbose=2,
                                n_jobs=-1)

    random.fit(X_train, y_train)
    model_param[name] = random.best_params_

for model_names in model_param:
    print(f"----- Best Params for {model_names} -----")
    print(model_param[model_names])

```

Fitting 3 folds for each of 6 candidates, totalling 18 fits

Fitting 3 folds for each of 100 candidates, totalling 300 fits

----- Best Params for KNN -----

```
{'n_neighbors': 2}
```

----- Best Params for RF -----

```
{'n_estimators': 70, 'min_samples_split': 2, 'max_features': 7, 'max_depth': 10}
```

```

[26]: ## Retraining the models with best parameters
models = {
    "Random Forest Regressor": RandomForestRegressor(n_estimators=100,
    ↪ min_samples_split=2, max_features=7, max_depth=None,
                                n_jobs=-1),
    "K-Neighbors Regressor": KNeighborsRegressor(n_neighbors=10, n_jobs=-1)
}
for i in range(len(list(models))):
    model = list(models.values())[i]
    model.fit(X_train, y_train)

    # make prediction
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    # evaluate the train and test dataset
    model_train_mae, model_train_mse, model_train_r2, model_train_rmse = ↪
    ↪ evaluate_model(y_train, y_train_pred)

    model_test_mae, model_test_mse, model_test_r2, model_test_rmse = ↪
    ↪ evaluate_model(y_test, y_test_pred)

    print(list(models.keys())[i])
    print('Model performance for training set')

```

```

print("- Root Mean Squared Error: {:.4f}".format(model_train_rmse))
print("- Mean Absolute Error: {:.4f}".format(model_train_mae))
print("- Mean Squared Error: {:.4f}".format(model_train_mse))
print("- R2 Score: {:.4f}".format(model_train_r2))

print("-----")

print('Model performance for testing set')
print("- Root Mean Squared Error: {:.4f}".format(model_test_rmse))
print("- Mean Absolute Error: {:.4f}".format(model_test_mae))
print("- Mean Squared Error: {:.4f}".format(model_test_mse))
print("- R2 Score: {:.4f}".format(model_test_r2))

print('='*35)
print('\n')

```

Random Forest Regressor

Model performance for training set

- Root Mean Squared Error: 140206.6656
- Mean Absolute Error: 39725.9240
- Mean Squared Error: 19657909076.0765
- R2 Score: 0.9760

Model performance for testing set

- Root Mean Squared Error: 231308.6359
- Mean Absolute Error: 101786.3822
- Mean Squared Error: 53503685047.8965
- R2 Score: 0.9289

=====

K-Neighbors Regressor

Model performance for training set

- Root Mean Squared Error: 373792.4427
- Mean Absolute Error: 104054.8716
- Mean Squared Error: 139720790254.0095
- R2 Score: 0.8295

Model performance for testing set

- Root Mean Squared Error: 297848.3225
- Mean Absolute Error: 121182.8125
- Mean Squared Error: 88713623204.0712
- R2 Score: 0.8820

=====